LLMによる脆弱性検知における検知結果と説明内容の整合性分析

味谷 圭悟[1]　佐藤 将也[2,a)]

概要：LLM によるソースコードの脆弱性検知が研究されている．これらの研究では，主に LLM による脆弱性検知の性能を評価している．しかし，LLM が脆弱性の有無を正しく検知した場合であっても，脆弱性の原因や脆弱性タイプを正しく判定できているとは限らない．本稿では，脆弱性のあるソースコードを含むデータセットである Juliet Test Suite v1.3 for C/C++ を用い，脆弱性検知の正誤や脆弱性タイプ判定の正誤が，ソースコード説明と整合しているか分析した結果を報告する．

キーワード：脆弱性検知，LLM，LLM-as-a-judge

Consistency Analysis between Vulnerability Detection Results and Source Code Explanations in LLM-Based Vulnerability Detection

Keigo Ajitani[1]　Masaya Sato[2,a)]

Abstract: Vulnerability detection in source code using large language model (LLM) has been widely studied. However, most of these studies primarily evaluate the correctness of LLM-based vulnerability detection, whether the presence or absence of a vulnerability is correctly identified. Even when an LLM correctly detects a vulnerability, it remains unclear whether the model correctly understands the specific code responsible for the vulnerability or correctly identifies its vulnerability type. In this paper, using the Juliet Test Suite v1.3 for C/C++, a dataset of vulnerable source code, we report an analysis of whether the correctness of vulnerability detection and vulnerability type classification is consistent with the LLM's explanations of the source code.

1. はじめに

オープンソースソフトウェア（OSS）は，様々なソフトウェアの開発に広く利用されている．OSS ではソースコードが公開されており，その内容を直接解析することが可能である．広く利用される OSS に含まれる脆弱性は多くのソフトウェアに影響を及ぼす可能性があるため，その脆弱性を検知することは重要である．このため，ソースコードを起点としてソフトウェアの脆弱性を検知する手法が研究されている．

ソースコードに着目して脆弱性を検知する手法として，コード中の字句特徴に着目し，脆弱なコード片に類似するコード片を検出する手法 [1] や，抽象構文木の編集距離に着目して類似するセキュリティパッチを探索する手法 [2] がある．これらの字句や構造の特徴に着目した脆弱性検知手法に加え，大規模言語モデル（Large Language Model，以降 LLM）を活用した脆弱性検知手法が研究されている．しかし，脆弱性の原因や脆弱性タイプなどの周辺情報も含めて正しく判定できているかの評価は十分ではない．

LLM による脆弱性検知では，主に脆弱性を検知する能力の評価 [3] や，検知の能力を向上させる手法について研究されている [4]．一方，LLM が脆弱性の有無を正しく判定できたとしても，脆弱性の原因や脆弱性タイプを正しく

[1] 岡山県立大学大学院 情報系工学研究科
Graduate School of Computer Science and Systems Engineering, Okayama Prefectural University
[2] 岡山県立大学 情報工学部
Faculty of Computer Science and Systems Engineering, Okayama Prefectural University
[a)] masaya@c.oka-pu.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

判定できているとは限らない．例えば，ソースコード中の表面的な特徴や既知のコードパターンに基づき，脆弱性の有無のみを偶然正しく検知している可能性がある．この場合，実際とは異なる類似した脆弱性タイプに基づいて検知しており，検知結果に基づく対処が不適切になる可能性がある．さらに，検知結果自体は正しくとも，LLM が生成するソースコードの説明には，実際の処理内容や脆弱性の原因との不整合が生じる可能性がある．したがって，LLM による脆弱性検知の正誤だけでなく，どのような根拠に基づいて脆弱性の有無を判定しているのかを分析することが重要である．近年では OSS に含まれる脆弱性の報告が増加しており，迅速な検知と修正が求められている．こうした背景から，脆弱性検知の根拠や整合性を分析することは，OSS の脆弱性対策においても重要である．ただし，LLM に根拠を出力させると，今度は出力された根拠の正当性を検証する必要があり，容易には分析できない．

そこで本稿では，LLM による脆弱性の有無および脆弱性タイプの判定の正誤と，LLM が生成するソースコード説明の整合性との関係を定量的に分析する．LLM に脆弱性判定の根拠を直接説明させると，自身の判定に整合するもっともらしい根拠を事後的に生成する可能性がある．そこで，脆弱性の有無や脆弱性タイプを直接問うだけではなく，ソースコードの処理内容に関する説明を生成させる．そして，生成された説明が実際の処理内容とどの程度整合しているかを，脆弱性判定とは異なる観点の評価情報として用い，判定の正誤との関係を分析する．

実験では，脆弱性を含むソースコードのデータセットを対象として，LLM に脆弱性の有無および脆弱性タイプを判定させるとともに，ソースコードの処理内容に関する説明を生成させる．データセットの正解情報に基づいて各判定の正誤を評価し，生成された説明については，データセットに含まれる説明との整合性を評価する．また，多数のソースコードを効率的に評価するため，説明の整合性判定には LLM-as-a-judge を用いる．この方法により，53 種の脆弱性タイプを含む 780 件のデータを対象に評価し，脆弱性の有無および脆弱性タイプの判定の正誤とソースコートの内容に対する説明の整合性との関係を明らかにする．

本稿の貢献を以下に示す．

(1) 脆弱性検知に正答し，かつソースコードの安全性やセキュリティに対する説明が整合する場合は，独立であると仮定したときに期待される頻度よりも，統計的に有意に多く出現していた．このことから，脆弱性検知に正答したとき，ソースコードの安全性やセキュリティに対する説明の内容が整合する傾向にあることを明らかにした．

(2) 脆弱性タイプの判定結果とソースコード説明の整合性との関連は弱いことから，ソースコード説明が整合した場合でも，必ずしも脆弱性タイプの正誤を正しく判定できるわけではないことを明らかにした．

2. LLM による脆弱性検知

LLM によるソースコードを対象とした脆弱性検知の性能に着目した研究がある．Avishree ら [3] は，LLM に与えるプロンプト，検知に使用する LLM モデル，および検知対象の脆弱性タイプの 3 つの観点から，脆弱性検知の性能に差が生じることを示した．プロンプトの違いによる脆弱性検知の性能を比較すると，データフローの追跡に基づくプロンプトの導入により，脆弱性検知における F1 スコアが最大 0.18 向上することが示された．検知に使用する LLM モデルや検知対象の脆弱性タイプによる脆弱性検知の性能を比較すると，LLM モデルと検知対象の脆弱性タイプによって脆弱性検知の性能に差が見られた．LLM モデルとして Llama-3.1-70B，検知対象の脆弱性タイプとしてコマンドインジェクションの組み合わせにおいては，高い脆弱性検知の性能を示した．一方で，境界外読み込みや書き込みの検知においては，当時最新の GPT-4 でも，脆弱性検知の性能が 5 割程度まで低下することが示された．これらのことより，LLM による脆弱性検知の性能は，プロンプト，LLM モデル，および検知対象のソースコードといった複数の要素から影響を受けていることが示された．

このように，LLM による脆弱性検知の能力を分析する研究はあるものの，脆弱性検知結果がソースコードをどのように解釈し，どのような根拠に基づいているかの検証は十分ではない．上記の手法では，脆弱性検知結果とモデルの判断根拠について，限られたサンプルの手動分析から得られた考察を述べるに留まっており，データセット全体を通した分析や定量的な評価は十分ではない．

3. 目的

本稿では，LLM による脆弱性検知や脆弱性タイプ判定の正誤がソースコード説明の整合性と関連があるかを定量的に分析する．ここで，ソースコード説明とは，ソースコードの処理や内容について，観点ごとに 1 から 2 文程度の自然言語で説明したものを指す．ソースコード説明の内容が一致しているは否かは，比較観点ごとに LLM-as-a-judge を用いて判定する．

本稿では，以下のリサーチクエスチョン（以降，RQ）を設定する．

RQ　LLM による脆弱性の有無および脆弱性タイプの判定の正誤は，ソースコード説明の整合性と関連するか

脆弱性の有無および脆弱性タイプの判定について，判定の正誤とソースコード説明の整合性との関連をそれぞれ分析する．これにより，正しい判定がソースコードの処理内容と整合する説明を伴う傾向にあるかを明らかにする．

© 2026 Information Processing Society of Japan

4. LLM-as-a-judge

LLM-as-a-judge とは，LLM が出力した回答の内容や品質を評価するために，別の LLM を評価者として用い，出力結果を審査，判定させる手法である [5]．人手による評価と比較して，大量の出力結果を効率的かつ一定の基準に基づいて評価できるという利点があり，LLM の性能評価や生成結果の比較などに利用されている．

LLM-as-a-judge における評価の方法には，いくつかの方法が存在する．代表的な方法に，スコアによる評価と，二値による評価がある．前者は，評価対象を離散的なスコアによって評価する方法であり，後者は，Yes/No などの二値で評価する方法である．前者は品質の細かな差を評価でき，文章の要約や品質評価の目的に適している．後者は評価が厳格であり，記述内容の事実や正誤を厳格に判断する目的に適している [5]．

5. 提案手法

5.1 提案手法の全体像

図 1 に提案手法の全体像を示す．提案手法では，LLM による脆弱性検知と脆弱性タイプ判定の正誤に対し，ソースコード説明の整合性との関連を分析する．手法は以下に示す 4 つの段階によって構成する．

1 段階　LLM による脆弱性検知

データセットから取得した脆弱性を含むソースコードに対して，脆弱性の有無，脆弱性が検知された場合においては対応する脆弱性タイプの判定，および，ソースコードの説明を生成する．

2 段階　LLM-as-a-judge によるソースコード説明の整合性判定

LLM の生成したソースコード説明とデータセットのソースコード説明の内容が一致するかを LLM-as-a-judge によって判定する．

3 段階　脆弱性検知正誤とソースコード説明の整合性の関連分析

脆弱性検知の正誤とソースコード説明の整合性判定結果の間に関連があるか分析する．ここで，脆弱性検知に正答したとき，ソースコード説明が説明が整合する傾向にあれば，脆弱性検知はソースコード説明の整合性を伴っていると判断できる．このことから，脆弱性検知の正誤に対し，ソースコード説明の整合性がどのような結果を取る傾向にあるかを分析する．

4 段階　脆弱性タイプ判定の正誤とソースコード説明の整合性の関連分析

脆弱性タイプ判定の正誤とソースコード説明の整合性判定結果の間に関連があるか分析する．3 段階と同様に，脆弱性タイプ判定の正誤に対し，ソースコード説明の整合性がどのような結果を取る傾向にあるかを分析する．

表 1　Juliet Test Suite のテストケースから取得可能な項目

項目	内容
ファイル名	テストケースの名前
脆弱性タイプ	対応する脆弱性タイプ
source の説明	データを生成や取得するソースコード処理の説明
sink の説明	データを利用するソースコード処理の説明
flaw の説明	プログラムの正しさ，安全性，およびセキュリティに影響を与えるソースコード処理の説明
Flaw Variant	制御やデータフローの違いによるテストケースの分類
脆弱性を含むソースコード	脆弱性を含むソースコード関数
脆弱性修正済みのソースコード	脆弱性修正済みのソースコード関数

5.2 脆弱性を含むソースコードのデータセット

Juliet Test Suite v1.3 for C/C++[6]（以降，Juliet Test Suite）には，C と C++を対象に，特定の脆弱性タイプごとのテストケース 64,099 件が記録されている．各テストケースには，特定タイプの脆弱性を含むソースコード，その脆弱性を修正したソースコード，脆弱性タイプ，ソースコード内容の説明などが記録されている．**表 1** に，Juliet Test Suite のテストケースから取得できる項目を抜粋する．検証には以下の 5 つの条件を満たすテストケースを用いる．

条件 1　脆弱性を含むこと

LLM が脆弱性を検知できるかどうか検証するために，脆弱性を含むソースコードを使用する．

条件 2　C で記述されていること

報告された脆弱性は C で記述されている場合が多い．このため，C で記述されたソースコードを使用する．

条件 3　複数の脆弱性タイプが混在しないこと

Juliet Test Suite には，一つのテストケースに対して単一の脆弱性タイプが割り当てられている場合と，複数の脆弱性タイプが割り当てられている場合がある，検証では，脆弱性タイプの正誤を，正解ラベルとの一致によって判定する．このため，複数の脆弱性タイプが割り当てられているテストケースは除外し，単一の脆弱性タイプのみが割り当てられているテストケースを使用する．

条件 4　ソースコードの説明が記録されていること

Juliet Test Suite には，ソースコード説明として，データを生成または取得する処理（以降，source），データを利用する処理（以降，sink），プログラムの正しさ，

© 2026 Information Processing Society of Japan

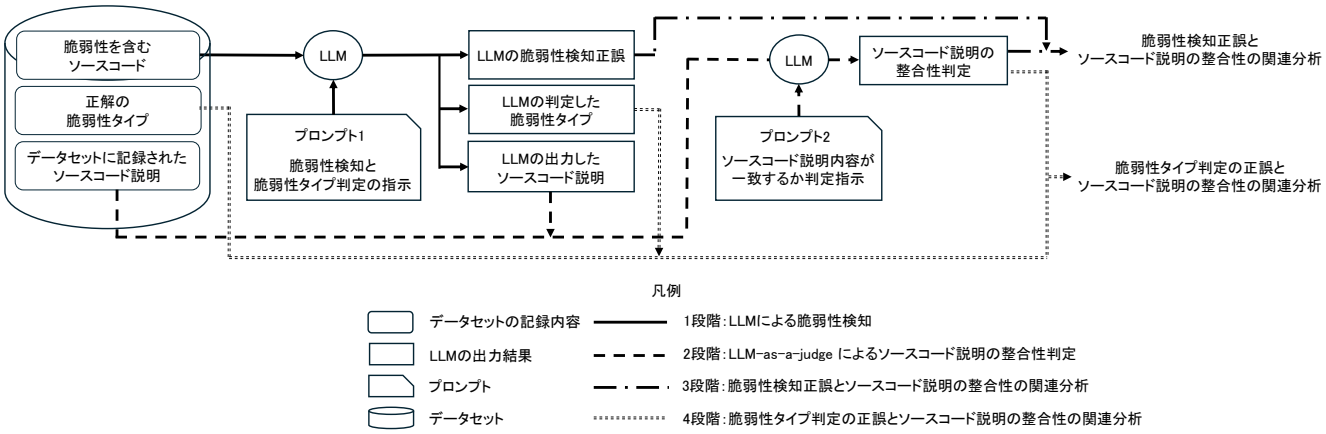


図 1　LLM による脆弱性検知と検知結果の分析

安全性，またはセキュリティに影響を与える可能性のある処理（以降，flaw）の 3 つの観点でソースコード説明が記録されている．各ソースコードに対して 3 つの観点すべての説明が必ず記録されてはおらず，説明が欠損している場合がある．分析において，分析対象のソースコードによってソースコード説明の有無に差があると比較ができないため，source，sink，および flaw の 3 項目の説明がすべて記録されているテストケースのみを使用する．

条件 5　当該脆弱性タイプにおける基本的な脆弱性パターンを含むこと

Juliet Test Suite のテストケースは，制御やデータフローなど，ソースコードの複雑さの違いによって，Flow Variant と呼ばれるグループに分類されている．このうち，Flow Variant が Baseline に分類されるテストケースは，各脆弱性タイプに応じて基本的な構造を含んでいる．こうしたテストケースは，複数の関数やファイルをまたぐような複雑な構造を持つことが少なく，ソースコード中の脆弱性の原因となる箇所が，単一の関数やファイル内で完結する場合が多い．このため，脆弱性を含むソースコードの抽出や，脆弱性検知の実験が容易である．以上の理由から．本検証では Flow Variant が Baseline に分類されたテストケースを使用する．

上記の条件を基に脆弱性を含むソースコードを抽出した，抽出できたソースコードは 780 件，割り当てられた脆弱性タイプは 53 種類である．**図 2** に，脆弱性タイプごとのソースコード件数の分布を示す．ただし，抽出できた脆弱性を含むソースコードの件数が 10 件以上の脆弱性タイプ 25 種類のみを示し，残りの脆弱性タイプ 28 種類はその他とした．

検証では，脆弱性検知時にヒントとなりうる記述を置換や除去する前処理を適用する．Juliet Test Suite では，脆弱性を含む関数に対して，脆弱性タイプの名称が記述されている．このため，関数名を“func”のように意味のない文字列に置換した．また，コメントはすべて除去した．

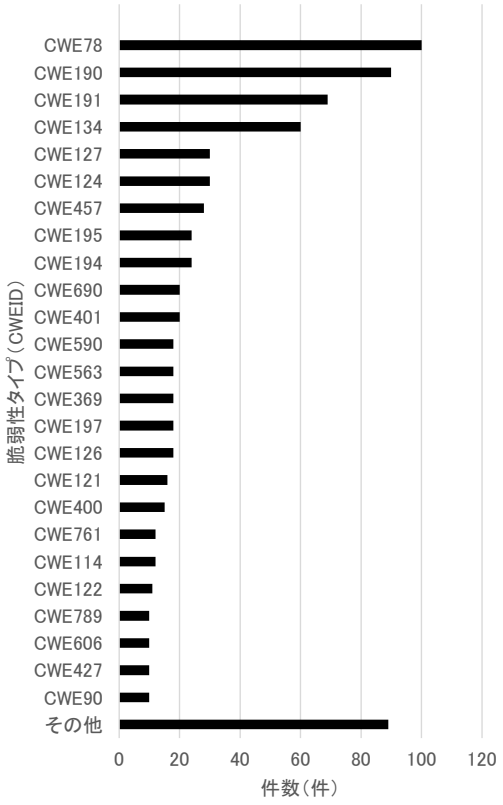


図 2　脆弱性タイプごとのソースコード件数の分布

5.3　LLM による脆弱性の検知

LLM によるソースコードの脆弱性検知方法を述べる．脆弱性の検知にはローカル LLM を用いる．使用する LLM モデルは，動作に必要な GPU メモリなどの計算機資源を考慮した上で，実験的に gpt-oss:20b [7] を採用する．以下に入力とプロンプト指示の内容を示す．**表 3** に LLM に出力させる項目を示す．

- 入力

© 2026 Information Processing Society of Japan

表 2 脆弱性の検知において出力させる項目

項目	内容
vulnerability_prediction	脆弱性有無の判定結果 (Vulnerable または NotVulnerable で判定)
predict_cwe	脆弱性タイプの判定結果 (CWEID で判定)
source_description	source に対するソースコード説明
sink_description	sink に対するソースコード説明
flaw_description	flaw に対するソースコード説明

脆弱性検知の対象として，5.2 節で示した脆弱性を含むソースコード 780 件を使用する．

- プロンプト
  各ソースコードに対して，脆弱性の検知と，source，sink，flaw の各観点についてソースコードの説明の生成を指示する．また，脆弱性が検知された場合においては，脆弱性タイプの判定を指示する．プロンプトの全文を**図 A·1** に示す．

5.4 LLM-as-a-judge によるソースコード説明の整合性判定

LLM-as-a-judge によるソースコード説明の整合性判定方法を述べる．判定に利用するモデルは gpt-oss:20b である．以下に，入力とプロンプト指示の内容を示す．表 3 に LLM に出力させる項目を示す．

- 入力
  ソースコード説明の整合性判定の対象として，5.3 節で示した LLM の生成した source，sink，flaw の各観点のソースコード説明と，データセットに記録されている対応するソースコード説明のペアを使用する．
- プロンプト
  LLM の生成したソースコード説明と，データセットに記録された対応するソースコード説明のペアについて，説明の内容が一致しているかの判定を指示する．判定は，source，sink，flaw の各観点について，三段階のスコアによって判定する．プロンプトの全文を**図 A·2** に示す．

ソースコード説明の整合性判定として，ソースコード説明の意味が一致しているかを評価する．ここで，意味が一致するとは，単語や語順ではなく，説明の意味として一致すればよいものとする．ただし，意味の一致には，一部が一致している場合や表現や単語は異なるが内容としては一致している場合など，一致度合いに差がある可能性がある．このことから，一致度を段階的に判定できるよう，以下のスコアにより判定する．

スコア 1　ソースコード説明の内容が一致する

スコア 2　ソースコード説明の内容が部分一致する

表 3 ソースコード説明の整合性判定において生成させる内容

項目	生成する内容
score_source	source 説明の内容が一致するかのスコア
score_sink	sink 説明の内容が一致するかのスコア
score_flaw	flaw 説明の内容が一致するかのスコア

表 4 ソースコード説明の整合性と脆弱性検知正誤に対する χ^2 検定

観点	χ^2 検定量	自由度	p 値	Cramer's V
source	89.017	2	4.681×10^{-20}	0.338
sink	129.174	2	8.915×10^{-29}	0.407
flaw	561.562	2	1.144×10^{-122}	0.848

スコア 3　ソースコード説明の内容が不一致する

5.5 脆弱性検知の正誤とソースコード説明の整合性判定結果の関連分析

脆弱性検知結果の正誤と，source，sink，および，flaw の 3 観点それぞれのソースコード説明の整合性判定結果との関連性を分析する．ソースコード説明の整合性，脆弱性検知結果，および脆弱性タイプ判定の正誤は，いずれもカテゴリ変数であることから，χ^2 検定を用いてカテゴリ変数間の関連の有無を検定する．

χ^2 検定は，カテゴリ変数間の独立性を検定する手法である．変数間が独立である，すなわち両変数の間に関連がないと仮定した場合に期待される頻度と，実際に観測された頻度との差を，χ^2 統計量として算出する．この統計量が，変数間が独立であると仮定した場合に偶然生じる範囲を超えて大きいかどうかを，有意水準と p 値の比較によって判断する．これにより，両者の関連が，偶然の変動では説明しにくいものであるかを判断することができる．

χ^2 検定の結果を基に，脆弱性検知の正誤によって，ソースコード説明の整合性がどのような結果をとる傾向にあるか分析する．

5.6 脆弱性タイプ判定の正誤とソースコード説明の整合性判定結果の関連分析

5.5 節と同様に，脆弱性タイプ判定の正誤と，source，sink，および，flaw の 3 観点それぞれのソースコード説明の整合性判定の結果に対して χ^2 検定する．

6. 結果

6.1 脆弱性検知の正誤とソースコード説明の整合性

脆弱性検知の正誤と source，sink，および，flaw の 3 観点それぞれのソースコード説明の整合性判定の結果に対して，χ^2 検定をした結果を**表 4** に示す．

統計的な関連を判断する基準として有意水準を 0.05 に設定する．p 値が有意水準を下回っていた場合，統計的に関連があると判断できる．表 4 より，いずれの観点においても $p < 0.05$ を満たしている．このことより，脆弱性検

© 2026 Information Processing Society of Japan

source説明の一致判定 ＼ 脆弱性検知	検知:成功	検知:失敗
説明：一致	4.973	-4.973
説明：部分一致	2.870	-2.870
説明：不一致	-9.339	9.339

sink説明の一致判定 ＼ 脆弱性検知	検知:成功	検知:失敗
説明：一致	7.887	-7.887
説明：部分一致	3.294	-3.294
説明：不一致	-11.35	11.35

flaw説明の一致判定 ＼ 脆弱性検知	検知:成功	検知:失敗
説明：一致	15.09	-15.09
説明：部分一致	4.011	-4.011
説明：不一致	-23.68	23.68

図 3　ソースコード説明の整合性と脆弱性検知正誤に対する残差検定

source説明の一致判定 ＼ 脆弱性タイプ判定	判定:成功	判定:失敗
説明：一致	4.022	-4.022
説明：部分一致	-2.202	2.202
説明：不一致	-3.308	3.308

sink説明の一致判定 ＼ 脆弱性タイプ判定	判定:成功	判定:失敗
説明：一致	4.360	-4.360
説明：部分一致	-3.835	3.835
説明：不一致	-1.367	1.367

flaw説明の一致判定 ＼ 脆弱性タイプ判定	判定:成功	判定:失敗
説明：一致	3.835	-3.835
説明：部分一致	-2.929	2.929
説明：不一致	-3.611	3.611

図 4　ソースコード説明の整合性と脆弱性タイプ判定の正誤に対する残差検定

知の正誤と，source，sink，および，flaw の 3 観点それぞれのソースコード説明の整合性には，統計的な関連があると判断できる．

source，sink，および，flaw の各観点のソースコード説明の整合性判定結果のうち，どの観点の説明の整合性が，脆弱性検知の正誤との関連が強いか比較する．比較には，Cramer's V を用いる．Cramer's V は，χ^2 統計量，検定する対象データの件数，および自由度を用いて算出される指標であり，以下に示す式によって定義される．n はデータ件数，r はデータをクロス表として集計した際の行数，c は列数を示す．

$$V = \sqrt{\frac{\chi^2}{n \cdot \min(r-1, c-1)}}$$

Cramer's V 値は，0 から 1 の値を取る，値が 0 に近いほど統計的な関連が弱く，1 に近いほど統計的な関連が強いことを示す．Cramer's V 値を比較すると，flaw，sink，source のソースコード説明の整合性の順に，脆弱性検知結果の正誤と統計的な関連が大きいことがわかる．

最後に，残差分析によってどの変数の組み合わせにおいて有意な偏りが見られるか分析する．残差分析は，各変数に対して観測度数と期待度数の差（以降，残差）を元に算出した調整済み標準化残差を用い，各変数の値が統計的にどの程度偏っているか判定する手法である．調整済み標準化残差は，近似的に標準正規分布に従う．調整済み標準化残差の値が正の値を取る場合，期待される頻度よりも多く出現していることを示す．負の値を取る場合，期待される頻度よりも出現が少ないことを示す．本稿では有意水準を 5%とし，調整済み標準化残差の絶対値が 1.96 以上となる変数の組み合わせを，有意な偏りであると見なす．

残差分析の結果を**図 3** に示す．図 3 より，いずれの組み合わせも，調整済み標準化残差の絶対値が 1.96 を上回っている．このことから，いずれの組み合わせにおいても統計的な偏りがあると言える．一方で，ソースコード説明の一致と脆弱性検知の成功において，調整済み標準化残差の値が正の方向に大きい傾向が見られる．このことより，ソースコード説明の一致と脆弱性検知の成功の組み合わせは，期待される頻度よりも多く出現しており，統計的な偏りが特に大きい傾向があることがわかる．

以上のことより，LLM による脆弱性検知の正誤とソースコード説明の整合性判定の結果に統計的な関連があることがわかった．関連の強さを分析すると，脆弱性検知の正誤と flaw 説明の整合性について，Cramer's V 値が 0.848 を取っており，sink や source 説明に比べて関連が強いことがわかる．さらに残差検定の結果から，脆弱性検知に正答し，かつ flaw 説明が一致する場合において，残差の値が最も大きいことがわかる．このことから，脆弱性検知に正答したときに，flaw 説明が整合する傾向にあることがわかる．

表 5　ソースコード説明の整合性と脆弱性タイプ判定の正誤に対する χ^2 検定

観点	χ^2 検定量	自由度	p 値	Cramer's V
source	18.059	2	1.982×10^{-4}	0.169
sink	21.352	2	2.310×10^{-5}	0.184
flaw	22.934	2	1.047×10^{-5}	0.190

6.2　脆弱性タイプ判定の正誤とソースコード説明の整合性

脆弱性タイプ判定の正誤と，source，sink，および，flaw の 3 観点それぞれのソースコード説明の整合性判定の結果に対して，χ^2 検定をした結果を**表 5** に示す．いずれの観点においても $p < 0.05$ を満たしている．このことより，脆弱性タイプ判定の正誤と，source，sink，および，flaw の 3 観点それぞれのソースコード説明の整合性には，統計的な関連があると判断できる．

source，sink，および，flaw の各観点のソースコード説明の整合性判定結果のうち，どの観点の説明の整合性が，脆弱性タイプ判定の正誤との関連が強いか，Cramer's V 値によって比較する．Cramer's V 値を比較すると，6.1 節で比較した脆弱性検知の正誤と同様に，flaw，sink，source のソースコード説明の一致評価の順に，脆弱性タイプ判定結果と統計的な関連があることがわかる．一方で，脆弱性検知の正誤に対する分析と比べて，Cramer's V の値が小さい．source，sink，および，flaw のいずれの観点に対しても，0.2 を下回っている．このことより，観点ごとの差

© 2026 Information Processing Society of Japan

は小さく，どの観点においても関連は弱いことがわかる．特に，flaw 説明の整合性においては，脆弱性検知の正誤に対する分析結果と比べて，0.658 ポイントと大幅に低下している．以上のことより，脆弱性検知結果の正誤と比べると，脆弱性タイプ判定の正誤とソースコード説明の整合性の関連は弱いことがわかる．

残差分析によってどの変数の組み合わせにおいて有意な偏りが見られるか分析する．残差分析の結果を**図 4** に示す．図 4 より，一部の組み合わせにおいては調整済み標準化残差の絶対値が 1.96 を下回っている．一方で，ソースコード説明が一致し，かつ脆弱性タイプ判定に正答している組み合わせにおいては，調整済み標準化残差の値が正の方向に大きい傾向が見られる．

以上のことより，LLM による脆弱性タイプ判定の正誤とソースコード説明の整合性判定の結果には，統計的な関連があることがわかった．一方，関連の強さを比較すると，Cramer's V 値の比較結果に基づき，関連は弱いことがわかった．この結果から，ソースコード説明の整合性が脆弱タイプの正しい判定に寄与する程度は，脆弱性検知への寄与に比べて限定的である可能性が考えられる．また，脆弱性検知の正誤に対する分析結果と比べて，flaw 説明の整合性との関連が大幅に低下しているという結果から，ソースコードの安全性やセキュリティ欠陥に関する説明が整合し，かつ脆弱性の有無を正しく判定できたとしても，必ずしも脆弱性タイプの正誤を正しく判定できるわけではないことがわかる．

7. おわりに

LLM による脆弱性検知と脆弱性タイプ判定の正誤について，ソースコードに対する説明との整合性の観点から分析した．分析の結果，脆弱性検知の正誤とソースコード説明の整合性には，source，sink, および flaw のすべての観点に対して統計的な関連があった．特に，脆弱性検知に正答し，かつソースコードの安全性やセキュリティに対する説明の内容が整合する場合は，独立であると仮定したときに期待される頻度よりも，統計的に有意に多く出現していた．このことから，脆弱性検知に正答したとき，ソースコードの安全性やセキュリティに対する説明の内容が整合する傾向にあることを明らかにした．一方，脆弱性タイプ判定の正誤とソースコード説明の整合性は，関連が弱いことがわかった．このことより，ソースコード説明が整合し，かつ脆弱性の有無を正しく判定できたとしても，必ずしも脆弱性タイプの正誤を正しく判定できるわけではないことを明らかにした．

残された課題として，異なる LLM モデルを使用した場合の脆弱性検知結果の比較，および，LLM が脆弱性タイプ判定を誤った場合において，どのような脆弱性タイプへの誤答が多いか分析することが挙げられる．

謝辞 本研究の一部は，JST 経済安全保障重要技術育成プログラム【K Program】Japan Grant Number【JPMJKP24K2】による助成を受けたものです．

参考文献

[1] 荒川玲佳, 鐘本 楊, 塩治榮太朗, 秋山満昭：集合間類似度算出アルゴリズムを用いたソフトウェアに内在する脆弱なコードのクローン検知手法，信学技報， Vol. 123, No. 86, pp. 32–39 (2023).
[2] 佐藤将也：マルチビュー ANN 探索と AST 編集距離によるセキュリティパッチ探索，信学技報， Vol. 125, No. 223, pp. 86–93 (2025).
[3] Khare, A., Dutta, S., Li, Z., Solko-Breslin, A., Alur, R. and Naik, M.: Understanding the Effectiveness of Large Language Models in Detecting Security Vulnerabilities, *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*, pp. 103–114 (2025).
[4] Du, X., Zheng, G., Wang, K., Zou, Y., Wang, Y., Deng, W., Feng, J., Liu, M., Chen, B., Peng, X., Ma, T. and Lou, Y.: Vul-RAG: Enhancing LLM-based Vulnerability Detection via Knowledge-level RAG, *ACM Trans. Softw. Eng. Methodol.* (2026).
[5] Gu, J., Jiang, X., Shi, Z., Tan, H., Zhai, X., Xu, C., Li, W., Shen, Y., Ma, S., Liu, H., Wang, S., Zhang, K., Lin, Z., Zhang, B., Ni, L., Gao, W., Wang, Y. and Guo, J.: A survey on LLM-as-a-judge, *The Innovation*, Vol. 7, No. 6, p. 101253 (online), DOI: https://doi.org/10.1016/j.xinn.2025.101253 (2026).
[6] National Institute of Standards and Technology: Juliet C/C++ 1.3 - NIST Software Assurance Reference Dataset, , available from ⟨`https://samate.nist.gov/SARD/test-suites/112`⟩ (accessed 2026-08-05).
[7] OpenAI: gpt-oss-120b & gpt-oss-20b Model Card, *arXiv preprint arXiv:2508.10925*, (online), available from ⟨https://arxiv.org/abs/2508.10925⟩ (2025).

付　　録

A.1 LLM に与えたプロンプト

LLM による脆弱性検知時に入力したプロンプトを図 A·1 に，LLM-as-a-judge によるソースコード説明の整合性判定時に入力したプロンプトを図 A·2 に示す．

© 2026 Information Processing Society of Japan

```
Please analyze whether the code contein Vulnerability.
# rule:
- Answer in "response schema format", include "required information" .
- Do not include anything other information.
- Do not omit items.
- Do not insert line blanks.
- Do not use single quotes, use double quotes.
# response schema format
    {schema}
# required information:
    * prediction
        - Vulnerability prediction. Answer in "Yes" or "No"
    * predict_cwe_id
        - If the code contains vulnerability, answer most related cweid
        - CWE<some digits>, where <some digits> is CWE number.
        - The number must be 1 to 4 digits long.
        - Do not add leading zeros.
        - Do not add hyphens between digits.
        - If the code does not contain vulnerability, answer "No CWE"
    * predict_reason
        - Why did you judge the source code is vulnerable, or not? Explain shortly.
    * predict_sink
        - Identify sinks.
        - Sink is the destination of a data flow where data is used.
        - Explain why this code location is a sink in one or two sentences.
        - If there are no sinks, answer "There are no sinks."
    * predict_source
        - Identify sources.
        - Source is the origin of a data flow.
        - Explain why this code location is a source in one or two sentences.
        - If there are no sources, answer "There are no sources."
    * predict_flaw
        - Identify flaws.
        - Include cartainly and potentially flows.
        - Flaw is an operation that affect the correctness, safety, or security of the program.
        - Explain why this code location is a flaw in one or two sentences.
        - If there are no flaws, answer "There are no flaws."
# Code snippet:
    {snippet}
```

図 A·1 LLM による脆弱性検知時に入力するプロンプト

```
I will provide three pairs of descriptions.
Your task is to evaluate whether the descriptions in each pair are semantically equivalent.
# Scoring criteria:
    1 = Same meaning. Only wording or stylistic differences.
    2 = Partially overlapping meaning. Some information is missing, added, or less precise.
    3 = Different or conflicting meaning.
    4 = If dataset description is empty. assign score 4
# Reason criteria:
    Explain briefly why the score was assigned.
# Definitions:
    - Source: the origin of a data flow.
    - Sink: the destination of a data flow where data is used.
    - Potential Flaw: an operation that may affect the correctness, safety, or security of a program.
# Instructions:
    - Focus only on semantic meaning.
    - Ignore differences in grammar, wording, writing style, or sentence structure unless they change the meaning.
    - Use the definitions only as background context.
    - Compare the two descriptions within each pair against each other, not against the definitions.
    - Result must follow Output schema in JSON format.
# Input:
    ## Pair1: Source
        source escription (predict):
        {predict_source}
        source description (dataset):
        {dataset_source}
    ## Pair2: Sink
        sink description (predict):
        {predict_sink}
        sink description (dataset):
        {dataset_sink}
    ## Pair3: Flaw
        flow description (predict):
        {predict_flaw}
        flaw description (dataset):
        {dataset_potential_flaw}
        {dataset_flaw}
# Output format:
    score_source: <1-4>
    reason_source: <short explanation>
    score_sink: <1-4>
    reason_sink: <short explanation>
    score_potential_flaw: <1-4>
    reason_potential_flaw: <short explanation>
# Output schema:
    {schema}
```

図 A·2 LLM-as-a-judge によるソースコード説明の整合性判定時に入力するプロンプト

© 2026 Information Processing Society of Japan